

ASSIGNMENTS – OOADI

LECTURE 6

-
1. To begin with we need a full framework of classes and interfaces. If you did not complete this last time around, you can make use of the code I have provided on Moodle. It is a minimal domain model that captures most of what is needed, but also not much more 😊

With the minimal setup we can use something like this to create a list of trains, to continue to work on.

```
package lecture6;

import java.util.*;
import static java.util.Arrays.asList;

public class FleetData {
    public static List<Train> sampleFleet() {
        List<Wagon> pax3 = asList(new Wagon(26, 40), new Wagon(26,40), new
Wagon(26,40));
        List<Wagon> goods4 = asList(new Wagon(14,15), new Wagon(14,15), new
Wagon(14,15), new Wagon(14,15));
        List<Wagon> mixed2 = asList(new Wagon(24,35), new Wagon(16,16));

        return new ArrayList<> (List.of(
            new DieselTrain ("D-101", goods4),
            new BatteryTrain("E-201", pax3, 75),
            new DieselTrain ("D-102", mixed2),
            new BatteryTrain("E-202", goods4, 15),
            new BatteryTrain("E-203", mixed2, 55)
        ));
    }
}
```

-
2. Sorting can be done like so

```
package lecture6;

import java.util.*;
import static java.util.Comparator.*;

public final class TrainSort {
    private TrainSort() {}

    public static final Comparator<Train> BY_TYPE
        = comparing(Train::getDriveType).thenComparing(Train::getId);
}
```

```

public static final Comparator<Train> BY_LENGTH
    = comparingDouble(Train::getTotalLengthM).thenComparing(Train::getId);

public static final Comparator<Train> BY_WEIGHT
    = comparingDouble(Train::getTotalWeightT).thenComparing(Train::getId);

public static void sortByType(List<Train> trains) { trains.sort(BY_TYPE); }
public static void sortByLength(List<Train> trains) { trains.sort(BY_LENGTH); }
public static void sortByWeight(List<Train> trains) { trains.sort(BY_WEIGHT); }
}

```

Usage would be something along these lines, where I ask for a train listing sorted by weight first and then by type.

```

List<Train> fleet = FleetData.sampleFleet();
TrainSort.sortByWeight(fleet);
fleet.forEach(System.out::println);
System.out.println();
TrainSort.sortByType(fleet);
fleet.forEach(System.out::println);

```

This would, in my example here then result in the following consol output

```

DieselTrain{id=D-102, type=DIESEL, len=60,0m, wt=131,0t}
BatteryTrain{id=E-203, type=ELECTRIC, len=60,0m, wt=131,0t}
DieselTrain{id=D-101, type=DIESEL, len=76,0m, wt=140,0t}
BatteryTrain{id=E-202, type=ELECTRIC, len=76,0m, wt=140,0t}
BatteryTrain{id=E-201, type=ELECTRIC, len=98,0m, wt=200,0t}

BatteryTrain{id=E-201, type=ELECTRIC, len=98,0m, wt=200,0t}
BatteryTrain{id=E-202, type=ELECTRIC, len=76,0m, wt=140,0t}
BatteryTrain{id=E-203, type=ELECTRIC, len=60,0m, wt=131,0t}
DieselTrain{id=D-101, type=DIESEL, len=76,0m, wt=140,0t}
DieselTrain{id=D-102, type=DIESEL, len=60,0m, wt=131,0t}

```

3. package lecture6;

```

import java.util.List;

public class AssignmentDemo {

    public static void main(String[] args) {

        List<Train> fleet = FleetData.sampleFleet();

        // Show sorting of the trains
        TrainSort.sortByWeight(fleet);
        fleet.forEach(System.out::println);
        System.out.println();
        TrainSort.sortByType(fleet);
    }
}

```

```
fleet.forEach(System.out::println);

// a) By type. Lists three trains
var electrics = TrainSearch.find(fleet, TrainQueries.byType(DriveType.ELECTRIC));
System.out.println("\n" + electrics);
// b) By length. Larger than 80m is asked so only one train is listed
var longOnes = TrainSearch.find(fleet,
TrainQueries.byLength(TrainQueries.Op.GT, 80));
System.out.println("\n" + longOnes);
// c) By weight. Heavier than 180t is asked so only one train is listed
var heavyOnes = TrainSearch.find(fleet,
TrainQueries.byWeight(TrainQueries.Op.GTE, 180));
System.out.println("\n" + heavyOnes);
// d) Has electric drive AND weight < 200t. Here two trains are listed
var efficient = TrainSearch.find(fleet,
TrainQueries.isElectric().and(TrainQueries.byWeight(TrainQueries.Op.LT, 200)));
System.out.println("\n" + efficient);
}
```
